ZGB Native Security and Compliance Evidence
Last updated: 2026-06-09
Public page: https://hanoutommyliu-source.github.io/zgb-native-legal/security/index.html

Zoom Beta Technical Design Evidence - ZGB Native

Last updated: 2026-06-09

ZGB Native, also known as Zoom Game Broadcaster, is a native macOS app that helps a Zoom host prepare timed broadcast prompts and send them to Zoom breakout rooms during a meeting. The current review version is `0.1.0` build `1`, bundle id `ai.openclaw.zoom-game-broadcaster-native`, minimum macOS `14.0`.

Required Evidence

SSDLC

Evidence document: `review/security/ssdlc.md`

Summary: ZGB Native uses a lightweight SSDLC covering security requirements, design review, secret-handling rules, implementation controls, verification gates, release/package review, and post-release vulnerability handling. Current controls include production OAuth token non-persistence in the macOS app, macOS Keychain token storage when the local backend OAuth helper is used, local-only backend binding, Zoom credential minimization, public HTTPS redirect/documentation pages, source-control exclusions for secrets and tokens, and scan/build verification before Marketplace review.

SAST Results

Evidence document: `review/security/scan-results-summary.md`

Raw artifact: `review/security/semgrep-sast-2026-05-29.json`

Summary: Semgrep `1.164.0` was run on 2026-05-29 against tracked Swift and JavaScript source with JavaScript and Swift rules. It scanned 10 targets, ran 70 rules, and reported 0 findings and 0 errors.

DAST Results

Evidence document: `review/security/scan-results-summary.md`

Raw artifact directory: `review/security/dast-2026-06-05/`

Summary: Dynamic checks were rerun on 2026-06-05 against the local backend started on `http://127.0.0.1:18787` with dummy non-secret credentials and a temporary macOS Keychain service, and against the public HTTPS ZGB Native pages. The backend health endpoint, unknown-route handling, local sign-out route, OAuth start redirect, public pages, and TLS 1.2 handshake were verified.

Privacy Policy

Public URL: `https://hanoutommyliu-source.github.io/zgb-native-legal/privacy/`

Local source: `review/privacy.md`

Summary: The Privacy Policy describes Zoom authorization information, meeting details, broadcast message text, queue timing, saved plans, recent broadcast records, local app configuration, local storage, production authorization token non-persistence inside the macOS app, local backend Keychain token handling when that helper is used, sharing with Zoom for user-directed meeting actions, retention,

 user choices, and contact path.

Additional Evidence

Security Policy

Evidence document: `review/security/security-policy.md`

Summary: Covers supported version, data protection, access control, transport se
curity, secrets management, and local logging/diagnostics posture.

Incident Management and Response Policy

Evidence document: `review/security/incident-management-response-policy.md`

Summary: Covers incident types, severity, containment, credential rotation, reme
diation, verification, documentation, and notification decision points.

Vulnerability Management Procedure

Evidence document: `review/security/vulnerability-management-procedure.md`

Summary: Covers intake, triage, remediation targets, verification, dependency ha
ndling, and tracking for security findings.

Infrastructure and Dependency Management Policy

Evidence document: `review/security/infrastructure-dependency-management-policy.
md`

Summary: Covers architecture, dependency inventory, npm audit/package-lock contr
ols, Zoom SDK vendor handling, public HTTPS page controls, local backend loopbac
k binding, and change-management expectations.

TLS 1.2 or Higher

Evidence document: `review/security/scan-results-summary.md`

Summary: Public ZGB Native pages are hosted on GitHub Pages over HTTPS. An OpenS
SL TLS 1.2 handshake to `hanoutommyliu-source.github.io:443` succeeded again on
2026-06-05. The local backend uses `http://127.0.0.1` only for loopback communic
ation on the user's Mac; external Zoom, documentation, privacy, terms, redirect,
 and package-download traffic use HTTPS.

Public Review URLs

- Privacy Policy: `https://hanoutommyliu-source.github.io/zgb-native-legal/priva
cy/`
- Terms of Use: `https://hanoutommyliu-source.github.io/zgb-native-legal/terms-o
f-use/`
- Documentation: `https://hanoutommyliu-source.github.io/zgb-native-legal/docume
ntation/`
- Review package: `https://github.com/hanoutommyliu-source/zgb-native-legal/rele
ases/download/v0.1.0-review/ZGB-0.1.0-review.zip`

--- SSDLC ---
ZGB Native Secure Software Development Life Cycle

Last updated: 2026-06-05

Scope

This SSDLC applies to ZGB Native, also known as Zoom Game Broadcaster, a native macOS app and local Node.js OAuth helper backend for preparing timed Zoom breakout-room broadcast messages.

Security Requirements

- The app requests only the Zoom permissions needed for user-directed meeting authorization and meeting SDK credential flow.
- OAuth client secrets, OAuth tokens, meeting passcodes, and Zoom SDK credentials must not be committed to source control, included in app packages, pasted into support tickets, or pasted into Zoom Marketplace review comments.
- The local OAuth helper binds to `127.0.0.1` only.
- Public pages used for Marketplace documentation, privacy, terms, and OAuth redirect forwarding must be served over HTTPS with TLS 1.2 or higher.
- Production authorization must not persist OAuth access or refresh tokens inside the macOS app.
- User-generated queues, saved plans, recent broadcasts, and local configuration are stored locally on the user's Mac unless explicitly sent to Zoom as part of a user-directed meeting action.

Design Review

Security-sensitive design changes are reviewed before release. Current review areas include:

- Zoom OAuth and Meeting SDK credential handling.
- Redirect URL behavior and forwarding from public HTTPS callback pages to the local backend.
- Production authorization behavior, optional local backend macOS Keychain token storage, and sign-out/deauthorization behavior.
- Minimizing data sent to Zoom and avoiding unnecessary external services.
- Packaging hygiene so review ZIPs do not contain `.env` files, OAuth tokens, local backend `.data`, DerivedData, or SDK secrets.

Implementation Controls

- Secrets live in local environment files excluded from GitHub.
- On macOS, the backend stores Zoom OAuth tokens in Keychain. File-based token storage is reserved for local diagnostics only.
- The backend exposes a small route surface: health, OAuth start/callback, sign-out, and join credential generation.
- The app keeps the Zoom integration behind `ZoomMeetingSDKAdapter` and `ZoomAuthBackendClient` boundaries.
- Error messages should not reveal secrets or raw token values.

Verification Gates

Before review or release, run the smallest meaningful verification set for the change:

- Swift/Xcode build or tests for app code changes.
- `npm run check` for backend syntax validation.
- SAST scan for tracked Swift and JavaScript source.
- Dependency audit for Node backend dependencies.
- DAST smoke checks against the local backend route surface and public HTTPS pages.
- TLS 1.2 handshake verification for public HTTPS pages.
- Secret hygiene review before packaging or publishing.

Release and Review

Release artifacts are prepared from known build outputs and documented with size and SHA-256 hash. The current Beta review package is documented in `review/marketplace-fields.md`. Marketplace reviewer responses must not include secrets, private meeting information, or raw OAuth tokens.

Post-Release Handling

Security findings are triaged under the vulnerability management procedure. Incidents involving exposed credentials, unauthorized access, or user data exposure are handled under the incident management and response policy.

--- Security Policy ---
ZGB Native Security Policy

Last updated: 2026-06-05

Supported Version

The current review version is ZGB Native `0.1.0` build `1` for macOS 14.0 or later.

Data Protection

- ZGB Native stores queue content, saved session plans, recent broadcast records, and app configuration locally on the user's Mac.
- The production authorization flow does not persist OAuth access or refresh tokens inside the macOS app.
- Optional local OAuth backend mode on macOS stores Zoom OAuth tokens in the user's macOS Keychain. File-based token storage is reserved for local diagnostics only and is not used for production review builds.
- Secrets and tokens are excluded from source control, review packages, screenshots, documentation, support messages, and Marketplace comments.
- Broadcast messages are sent to Zoom meeting participants only when the user chooses to send them through the app.

Access Control

- Zoom meeting access depends on the user's Zoom authorization and meeting permissions.
- Breakout-room broadcast behavior requires the signed-in Zoom user to have the required Zoom host, co-host, administrator, or equivalent permissions.
- Local OAuth helper routes bind to `127.0.0.1` and are intended for local app use on the user's Mac.

Transport Security

- External Zoom OAuth, Zoom API, Zoom Meeting SDK, GitHub Pages documentation, and review download traffic use HTTPS.
- Public ZGB Native legal/documentation/OAuth redirect pages support TLS 1.2 or higher.
- Local loopback traffic between the app and backend may use `http://127.0.0.1` because it does not leave the user's Mac.

Secrets Management

- `.env`, `.env` backups, backend `.data`, OAuth tokens, client secrets, app packages, DerivedData, and provisioning profiles are excluded from GitHub.
- The backend deletes any legacy `Backend/.data/token.json` file when saving or clearing a Keychain token.
- Raw secrets must not be printed in logs, pasted into review forms, or sent in support messages.

- If a credential is suspected to be exposed, revoke or rotate it in Zoom Market
place and invalidate locally stored tokens.

Monitoring and Logging

The current review build is a local macOS utility and does not operate a server-
side production service for user data. Troubleshooting should use visible app er
rors, local backend health checks, and non-secret diagnostic details such as app
 version, macOS version, and endpoint status.

Contact

Use the developer contact email listed on the Zoom Marketplace app profile for s
ecurity or privacy issues.

--- Incident Management and Response Policy ---
ZGB Native Incident Management and Response Policy

Last updated: 2026-06-05

Scope

This policy covers suspected or confirmed security incidents affecting ZGB Nativ
e, its local OAuth helper backend, public review/legal/documentation pages, revi
ew package distribution, or Zoom Marketplace configuration.

Incident Types

- Exposed OAuth client secret, Meeting SDK secret, OAuth token, refresh token, m
eeting passcode, or private credential.
- Unauthorized access to Zoom meeting functionality through the app.
- Review package containing unintended secret, token, local data, or private fil
e.
- Vulnerability in the app or backend that could expose local data or misuse Zoo
m permissions.
- Public documentation, redirect, or download page compromise.

Response Process

1. Triage the report and identify affected version, account, file, endpoint, or
package.
2. Contain the issue by removing exposed files, disabling public links, stopping
 affected local services, or revoking affected credentials.
3. Preserve minimum necessary evidence without copying raw secrets into tickets,
 chats, or public comments.
4. Eradicate the cause with code, configuration, packaging, or documentation cha
nges.
5. Rotate or revoke affected Zoom credentials and invalidate local tokens when c
redential exposure is possible.
6. Verify the fix with targeted build, scan, DAST, TLS, package, or manual check
s.
7. Document the incident, impact, timeline, fix, and follow-up actions.
8. Notify affected parties or platforms when required by law, platform policy, o
r reasonable security practice.

Severity Guidelines

- Critical: confirmed active exploitation, exposed production secrets, or unauth
orized Zoom account access.
- High: likely credential exposure, package shipped with tokens/secrets, or vuln
erability allowing unauthorized meeting actions.

- Medium: vulnerability requiring local access or unusual user action, or public page misconfiguration without secret exposure.
- Low: documentation gap, hardening opportunity, or non-exploitable scan finding.

Recovery

Recovery may include publishing a corrected package, updating public documentation, rotating Zoom Marketplace credentials, requiring users to reauthorize, or instructing users to delete local Keychain token items and any legacy local token files.

--- Vulnerability Management Procedure ---
ZGB Native Vulnerability Management Procedure

Last updated: 2026-05-29

Intake

Vulnerabilities may be identified through SAST, dependency audit, DAST, manual code review, user reports, Zoom Marketplace review, or developer testing.

Triage

Each finding is reviewed for:

- Affected component: macOS app, local backend, public page, review package, or Zoom Marketplace configuration.
- Exploitability and required access.
- Impact on secrets, OAuth tokens, meeting passcodes, local user data, Zoom authorization, or meeting actions.
- Whether the issue is already mitigated by local-only binding, user authorization, or Zoom platform controls.

Remediation Targets

- Critical: contain immediately and remediate before further distribution.
- High: remediate before release or Marketplace resubmission.
- Medium: remediate in the next practical update unless risk requires faster action.
- Low: track and fix opportunistically.

Verification

Fixes are verified with the relevant subset of:

- Xcode build/tests.
- `npm run check`.
- Semgrep SAST scan.
- `npm audit`.
- Local backend DAST smoke checks.
- TLS 1.2 handshake checks for public pages.
- Secret/package hygiene review.

Dependency Handling

Node dependencies are locked in `Backend/package-lock.json` and audited with `npm audit`. The current backend intentionally avoids third-party runtime npm dependencies. Zoom Meeting SDK updates are reviewed manually because the SDK is vendor-provided and not distributed through npm.

Tracking

Security findings, remediation decisions, scan artifacts, and verification notes are stored under `review/security/` or the project context file as appropriate.

--- Infrastructure and Dependency Management Policy ---
ZGB Native Infrastructure and Dependency Management Policy

Last updated: 2026-05-29

Architecture

ZGB Native consists of:

- A native macOS SwiftUI app.
- A local Node.js OAuth helper backend bound to `127.0.0.1`.
- Zoom OAuth, Zoom API, and Zoom Meeting SDK integrations.
- Public HTTPS GitHub Pages content for Privacy Policy, Terms of Use, documentation, and OAuth redirect forwarding.
- A public GitHub Release asset for the review ZIP.

ZGB Native does not operate a multi-tenant production backend for user data in the current review build.

Dependency Inventory

- macOS 14.0 or later.
- Xcode/Swift toolchain for native app builds.
- Node.js 20 or later for the local backend.
- Zoom macOS Meeting SDK vendor payload.
- GitHub Pages and GitHub Releases for public review/support artifacts.

Dependency Controls

- `Backend/package-lock.json` is retained for reproducible npm dependency auditing.
- `npm audit --omit=dev` is run for backend dependency vulnerability checks.
- The current backend has no third-party production npm dependencies.
- Vendor Zoom SDK payloads are excluded from GitHub and are handled as external vendor artifacts.
- App packages, DerivedData, `.env`, backend `.data`, OAuth tokens, and secrets are excluded from GitHub.

Infrastructure Controls

- Public pages are hosted over HTTPS and verified with HTTP status and TLS 1.2 checks.
- Public OAuth redirect pages forward the Zoom authorization query string to the local loopback backend and do not store tokens or secrets.
- Local backend routes are intended for the user's Mac and bind to `127.0.0.1`.

Change Management

Changes to OAuth redirect URLs, public review URLs, app packaging, dependency files, or Zoom SDK integration require documentation updates and targeted verification before Marketplace resubmission.

--- Scan Results Summary ---
ZGB Native SAST, DAST, Dependency, and TLS Evidence

Last updated: 2026-05-29

SAST

Tool: Semgrep `1.164.0`

Command:

```bash
PATH="/Users/hanoutommyliu/Library/Python/3.14/bin:$PATH" semgrep scan --config
p/javascript --config p/swift --json --output review/security/semgrep-sast-2026-
05-29.json --exclude Vendor --exclude review/marketplace-upload --exclude review
/marketplace-upload-previous-20260528 --exclude review/live-screenshots .
```

Result:

- Scan completed successfully.
- Rules run: 70.
- Targets scanned: 10 tracked Swift/JavaScript files.
- Findings: 0.
- Errors: 0.

Artifact: `review/security/semgrep-sast-2026-05-29.json`

Dependency Audit

Tool: npm audit

Command:

```bash
npm audit --json > ../review/security/npm-audit-2026-06-05.json
```

Result:

- Critical vulnerabilities: 0.
- High vulnerabilities: 0.
- Moderate vulnerabilities: 0.
- Low vulnerabilities: 0.
- Total vulnerabilities: 0.
- Current backend production dependency count: 0 third-party production packages
.

Artifact: `review/security/npm-audit-2026-06-05.json`

Backend Syntax Check

Tool: Node.js syntax check

Command:

```bash
npm run check
```

Result:

- `node --check server.js` passed.

DAST

Target:

- Local backend started on `http://127.0.0.1:18787` with dummy non-secret credentials and temporary macOS Keychain service `ai.openclaw.zgb.dast-20260605`.
- Public HTTPS ZGB Native Privacy, Terms, Documentation, Production OAuth callback, and Development OAuth callback pages.

Checks:

- `GET /health` returned `{"ok":true}`.
- Unknown path returned `{"error":"Not found."}`.
- `POST /auth/zoom/sign-out` returned `{"ok":true}`.
- `GET /auth/zoom/start` returned a Zoom OAuth `302` redirect using dummy non-secret credentials and the production HTTPS redirect URI.
- Public Privacy, Terms, Documentation, Production OAuth callback, and Development OAuth callback pages returned HTTP 200 on 2026-06-05.

Artifact directory: `review/security/dast-2026-06-05/`

TLS 1.2

Tool: OpenSSL

Target: `hanoutommyliu-source.github.io:443`

Command:

```bash
echo | openssl s_client -tls1_2 -servername hanoutommyliu-source.github.io -connect hanoutommyliu-source.github.io:443
```

Result:

- TLS 1.2 handshake succeeded on 2026-06-05.
- Certificate subject: `CN=*.github.io`.
- Issuer: Let's Encrypt `R12`.
- Certificate validity observed: 2026-04-06 through 2026-07-05.

Detailed output: `review/security/dast-2026-06-05/summary.txt`